

Tennis Platform - Arquitectura técnica

Documento de definicion del producto y la arquitectura

1. Arquitectura general

La solución inicial será:

Frontend Next.js/React/TypeScript/PWA

-> REST API

-> Java 21/Spring Boot

-> PostgreSQL

El backend será un Modular Monolith con arquitectura hexagonal en cada módulo.

2. Por qué Modular Monolith

No se necesitan microservicios porque:

- El producto tiene un único dominio principal.
- Las reservas necesitan consistencia transaccional.
- No hay necesidades de escalado independiente.
- El equipo se beneficia de un despliegue único.
- Kafka, service discovery y tracing distribuido introducirían complejidad innecesaria.

Los límites modulares permitirán extraer un módulo en el futuro si aparece una razón técnica real.

3. Módulos backend

identity

Registro, login, logout, refresh, verificación de email, recuperación de contraseña, roles, sesiones y ciclo de vida de usuarios.

teacher

Perfil, estado y zona horaria del profesor único.

student

Perfil del alumno y relación de alumno gestionado por el profesor.

availability

Reglas semanales, excepciones, bloqueos y evaluación de disponibilidad.

lesson

Clases individuales y grupales, horarios, capacidad, estados y asistencia.

booking

Reservas, capacidad, duplicados, solapamientos y cancelaciones.

administration

Consola administrativa, usuarios y configuración global, incluido el límite de alumnos.

calendar

Consulta agregada de disponibilidad, clases y reservas. No es propietario de los datos.

shared

Primitivas técnicas mínimas: identificadores, errores comunes, reloj e infraestructura compartida. No contiene lógica de negocio específica.

4. Dependencias permitidas

- identity no depende de módulos de negocio.
- teacher puede consultar identity.
- student puede consultar identity y teacher.
- availability puede consultar teacher.
- lesson puede consultar teacher y availability.
- booking puede consultar student y lesson.
- administration puede consultar identity, teacher y student.
- calendar solo usa interfaces públicas de consulta.

No se permiten accesos directos a repositorios, entidades JPA o adaptadores internos de otro módulo.

5. Arquitectura hexagonal

Cada módulo tendrá:

- domain: entidades, objetos de valor, reglas y excepciones.
- application: casos de uso y puertos.
- adapters/in: REST controllers y adaptadores de entrada.
- adapters/out: persistencia y servicios externos.
- configuration: ensamblado de dependencias.

El dominio no dependerá de Spring, JPA, REST ni PostgreSQL.

6. Estructura del proyecto

backend/

- identity/
- teacher/
- student/
- availability/
- lesson/
- booking/
- administration/
- calendar/
- shared/

Dentro de cada módulo:

- domain/
- application/port/in/
- application/port/out/
- application/service/
- adapters/in/web/
- adapters/out/persistence/
- configuration/

La organización por módulo es preferible a una estructura global de controllers/services/repositories porque mantiene juntas las reglas de cada contexto.

7. Modelo de aplicación

Los controllers solo traducen HTTP a comandos y respuestas.

Los casos de uso principales son:

- RegisterStudent.
- VerifyEmail.
- LoginUser.
- ResetPassword.
- ManageStudent.
- ConfigureWeeklyAvailability.
- CreateLesson.
- BookLesson.
- CancelBooking.

- CancelLesson.
- MarkAttendance.
- UpdateStudentLimit.

8. REST API inicial

Identity

- POST /api/v1/auth/register
- POST /api/v1/auth/login
- POST /api/v1/auth/logout
- POST /api/v1/auth/refresh
- POST /api/v1/auth/verify-email
- POST /api/v1/auth/forgot-password
- POST /api/v1/auth/reset-password
- GET /api/v1/me
- PATCH /api/v1/me

Teacher y students

- GET /api/v1/teacher/profile
- PATCH /api/v1/teacher/profile
- GET /api/v1/teacher/students
- POST /api/v1/teacher/students/{userId}/manage
- DELETE /api/v1/teacher/students/{userId}/manage

Availability

- GET /api/v1/teacher/availability
- PUT /api/v1/teacher/availability/weekly
- POST /api/v1/teacher/availability/exceptions
- DELETE /api/v1/teacher/availability/exceptions/{id}

Lessons y bookings

- GET /api/v1/calendar
- POST /api/v1/teacher/lessons
- GET /api/v1/lessons/{id}
- POST /api/v1/lessons/{id}/bookings
- GET /api/v1/bookings
- POST /api/v1/bookings/{id}/cancel
- POST /api/v1/teacher/lessons/{id}/cancel
- POST /api/v1/teacher/lessons/{id}/attendance

Administration

- GET /api/v1/admin/configuration
- PATCH /api/v1/admin/configuration/student-limit
- GET /api/v1/admin/users
- PATCH /api/v1/admin/users/{id}/status

9. DTOs

Los DTOs estarán separados del dominio.

Principales grupos:

- Authentication: registro, login, verificación, recuperación y sesión.
- Profile: perfil común y perfil de profesor/alumno.
- Availability: reglas y excepciones.
- Lesson: creación, modificación, consulta y asistencia.
- Booking: creación, consulta y cancelación.
- Administration: configuración y usuarios.

No se expondrán entidades JPA, contraseñas, tokens ni datos privados innecesarios.

10. Persistencia

Los repositorios serán puertos:

- UserRepository.
- TeacherProfileRepository.
- StudentProfileRepository.
- TeacherStudentAssociationRepository.
- AvailabilityRuleRepository.
- AvailabilityExceptionRepository.
- LessonRepository.
- BookingRepository.
- PlatformConfigurationRepository.

Las implementaciones JPA estarán en adapters/out/persistence de cada módulo.

11. Transacciones

Los límites transaccionales estarán en la capa de aplicación.

La reserva debe:

1. Bloquear la clase.

2. Comprobar estado y capacidad.
3. Comprobar duplicado.
4. Bloquear o serializar la comprobación del alumno.
5. Comprobar solapamientos.
6. Crear la reserva.

La base de datos y la aplicación deben proteger conjuntamente la consistencia.

12. Errores

Se utilizará Problem Details con códigos como:

- LESSON_FULL.
- LESSON_OVERLAP.
- BOOKING_ALREADY_EXISTS.
- STUDENT_NOT_MANAGED.
- CANCELLATION_WINDOW_EXPIRED.
- STUDENT_SCHEDULE_OVERLAP.

Mapeo general:

- 400: formato o validación.
- 401: no autenticado.
- 403: sin permisos.
- 404: no encontrado.
- 409: conflicto.
- 422: regla de negocio.
- 500: error inesperado.

13. Seguridad

- Spring Security.
- Access token corto.
- Refresh token rotatorio.
- Hash de refresh tokens.
- BCrypt o Argon2id.
- CORS restringido.
- Rate limiting para autenticación.
- Autorización por rol y propiedad.
- No usar localStorage para refresh tokens.

14. Observabilidad

- Spring Actuator.
- Health checks.
- Logs estructurados.
- Correlation ID.
- Métricas HTTP.
- Métricas de reservas, conflictos y cancelaciones.

15. Testing

- Unit tests de dominio sin Spring.
- Tests de aplicación con puertos simulados.
- Tests REST.
- Tests con PostgreSQL real mediante Testcontainers.
- Tests de concurrencia.
- ArchUnit para dependencias.
- Tests E2E con Playwright.

16. Configuración

Perfiles:

- local.
- test.
- staging.
- prod.

Variables sensibles fuera de Git:

- Base de datos.
- Secretos JWT.
- CORS.
- Email.
- Parámetros de negocio.

17. ADRs principales

- Modular Monolith.
- Arquitectura hexagonal.
- Módulos por capacidad de negocio.
- identity como contexto único para auth y usuarios.
- PostgreSQL y Liquibase.
- UTC e IANA timezones.

- Reservas automáticas.
- Control transaccional de concurrencia.
- Disponibilidad como restricción con override del profesor.
- Calendar como consulta agregada.
- Sin microservicios ni Kafka en el MVP.